

08 — Visibility Map

Module: PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 35 min

Table of Contents

1. [Learning Objectives](#)
 2. [Overview](#)
 3. [VM Structure: 2 Bits Per Heap Page](#)
 4. [All-Visible Bit: How It Is Set and Used](#)
 5. [All-Frozen Bit: How It Is Set and Used](#)
 6. [Index-Only Scans and the VM](#)
 7. [VACUUM Uses the VM to Skip Pages](#)
 8. [VM File Structure](#)
 9. [pg_visibility Extension](#)
 10. [ASCII Diagrams](#)
 11. [Live Observation Queries](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions](#)
 16. [Exercises with Solutions](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Describe the 2-bit-per-page structure of the visibility map.
 - Explain exactly when the all-visible and all-frozen bits are set and cleared.
 - Explain how index-only scans use the VM to avoid heap fetches.
 - Describe how VACUUM uses the VM to skip already-clean pages.
 - Use `pg_visibility` to inspect the VM for any table.
 - Diagnose performance problems related to VM staleness.
-

Overview

The **Visibility Map (VM)** is a compact 1-bit-per-page (now 2-bit-per-page since PG 9.6) structure that tracks the visibility state of each heap page. It lives in the `_vm` fork of a relation's data file.

The VM exists for two reasons:

1. **VACUUM efficiency** — If a page is marked "all-visible", VACUUM can skip it entirely during the heap scan phase. This dramatically reduces the amount of work VACUUM needs to do on stable tables.
 2. **Index-only scan efficiency** — If a page is marked "all-visible", an index-only scan can return the indexed column values without fetching the heap page (avoiding the main heap I/O entirely).
-

VM Structure: 2 Bits Per Heap Page

Since PostgreSQL 9.6, the VM stores **2 bits** per heap page:

Bit	Name	Meaning
Bit 0	all-visible	All tuples on this page are visible to every running transaction (i.e., no snapshots need to see old versions on this page).
Bit 1	all-frozen	All tuples on this page have been frozen (their <code>t_xmin = FrozenTransactionId</code>). No future VACUUM freeze pass needs to visit this page.

Important: all-frozen implies all-visible. A page can be all-visible without being all-frozen.

The VM file is stored in 8 KB pages, same as heap pages. Each VM page tracks:

```
BLCKSZ × 8 / 2 bits = 8192 × 4 = 32,768 heap pages per VM page
```

For a 10 GB table (1,310,720 heap pages), the VM is only about 40 VM pages = 320 KB. The VM is tiny.

All-Visible Bit: How It Is Set and Used

Setting the all-visible bit

VACUUM sets the all-visible bit for a page when it completes processing that page and finds:

- All remaining `ItemId` slots are either UNUSED or point to live tuples.
- All live tuples have `t_xmax = 0` (not deleted) or `t_xmax` from an aborted transaction (effectively live).
- The oldest `t_xmin` on the page is older than the oldest active transaction snapshot (`OldestXmin`). This means no running transaction can have a snapshot that needs to see a different version of any tuple on this page.

Clearing the all-visible bit

The all-visible bit is cleared (set to 0) when **any modification is made to the page**:

- Any INSERT, UPDATE, DELETE, or HOT_UPDATE that touches the page clears the bit immediately in memory.
- The WAL record for the modification includes a bit to clear the VM.

How INSERT uses the VM

When INSERT needs a page, it looks at the FSM for free space. If it is inserting into a new page, the page's all-visible bit starts as 0. When VACUUM confirms all tuples are visible, the bit is set.

All-visible and the Page Header

The page header (`PageHeaderData.pd_flags`) also has a `PD_ALL_VISIBLE` flag bit that mirrors the VM. When a page is marked all-visible in the VM, the page header bit is also set, so a backend that already has the page in `shared_buffers` can check the flag without consulting the VM file.

All-Frozen Bit: How It Is Set and Used

Setting the all-frozen bit

VACUUM FREEZE (or the freeze pass during aggressive autovacuum) sets the all-frozen bit for a page when:

- All tuples on the page have been frozen: `t_xmin = FrozenTransactionId` (2).

Using the all-frozen bit

During a future VACUUM freeze scan, any page with the all-frozen bit set is completely skipped — PostgreSQL knows there is nothing to freeze on that page. This makes freeze passes over large, stable tables extremely fast.

All-frozen and `relfrozenxid`

When VACUUM determines a page is all-frozen, it updates the all-frozen bit. After completing the full table scan, VACUUM advances `pg_class.relfrozenxid` to reflect the oldest non-frozen XID still in the table. The higher `relfrozenxid`, the less work future freeze passes need to do.

Index-Only Scans and the VM

An **index-only scan** reads an index to find matching tuples, then (ideally) returns the index key values without reading the heap. The catch: index entries do not contain MVCC information — the heap is needed to verify visibility.

The VM solves this: If the heap page containing the tuple is marked **all-visible**, the tuple is guaranteed visible to all transactions. The executor can skip the heap fetch and return the indexed value directly.

```
-- This index-only scan can skip heap if all_visible bits are set
EXPLAIN SELECT email FROM users WHERE id BETWEEN 1 AND 100;
/*
   Index Only Scan using users_pkey on users
   Index Cond: ((id >= 1) AND (id <= 100))
   Heap Fetches: 0    ← all VM bits were set; no heap access needed
*/
```

If VM bits are not set (e.g., right after a bulk INSERT before autovacuum has run), `Heap Fetches > 0` appears in the output.

```
-- Force fresh VM bits on a table
VACUUM users;
-- Then re-run the explain – Heap Fetches should drop to 0
```

VACUUM Uses the VM to Skip Pages

During the heap scan phase of VACUUM:

- For each heap page: if the VM all-visible bit is set, VACUUM **skips the page entirely**.
- Only pages with the all-visible bit clear are scanned for dead tuples.

This makes VACUUM on a mostly-stable table extremely fast. A table of 1 billion rows where 99.9% of pages are all-visible only needs to scan 0.1% of the heap.

During the freeze pass:

- If the all-frozen bit is set, the page is skipped even during an aggressive freeze scan.

VM File Structure

Relation file: base/16384/16385 (heap data)
VM fork: base/16384/16385_vm (visibility map)

VM file contains standard 8192-byte pages.
Each page stores $2 \text{ bits} \times 4 = 8 \text{ bit pairs per byte}$:
32,768 heap pages' visibility info per VM page

For a table with N heap pages:
VM file size = $\text{ceil}(N / 32768) \times 8192 \text{ bytes}$

The VM is tiny relative to the heap. A 100 GB table (13 million pages) needs only 400 VM pages (3.2 MB).

pg_visibility Extension

The `pg_visibility` extension (bundled with PostgreSQL) lets you inspect the VM for any table:

```
CREATE EXTENSION IF NOT EXISTS pg_visibility;

-- Check all-visible and all-frozen bits for first 20 pages
SELECT blkno, all_visible, all_frozen
FROM pg_visibility_map('orders')
LIMIT 20;

-- Count visible/frozen pages
SELECT
    count(*) AS total_pages,
    count(*) FILTER (WHERE all_visible) AS all_visible_pages,
    count(*) FILTER (WHERE all_frozen) AS all_frozen_pages,
    round(100.0 * count(*) FILTER (WHERE all_visible) / count(*), 1) AS visible_pct
FROM pg_visibility_map('orders');

-- Find all-visible pages where the heap page header disagrees (should be 0)
SELECT count(*) FROM pg_check_visible('orders');

-- Check VM consistency for entire table
SELECT * FROM pg_check_frozen('orders');
```

ASCII Diagrams

VM Bit Layout

Heap file: N pages (8 KB each)

Page 0 Page 1 Page 2 Page 3 Page 4 Page 5 Page 6 Page 7 ...

VM file: 1 byte covers 4 heap pages (2 bits each)

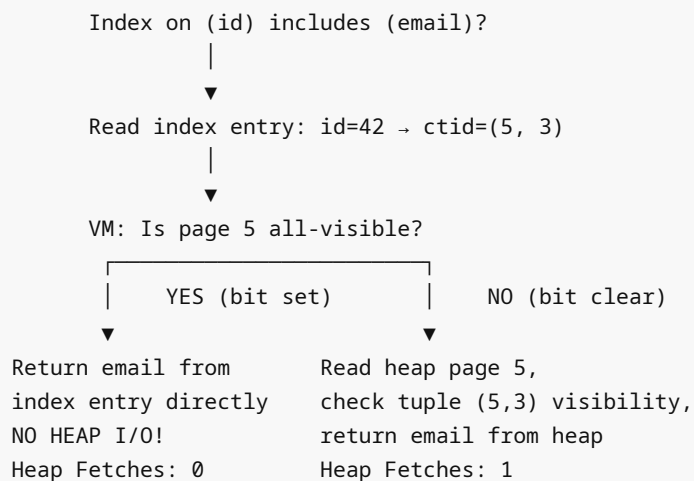
Byte 0:
bits 1-0: Page 0 (bit1=frozen, bit0=visible)
bits 3-2: Page 1 (bit1=frozen, bit0=visible)
bits 5-4: Page 2
bits 7-6: Page 3
Byte 1: Pages 4-7
...

Example byte value:

0b11_11_11_01 = pages 0-2 all-visible+all-frozen,
page 3 all-visible only (bit1=0, bit0=1)

Index-Only Scan with VM

Query: SELECT email FROM users WHERE id = 42



Live Observation Queries

```
-- 1. Setup
CREATE EXTENSION IF NOT EXISTS pg_visibility;

CREATE TABLE vm_demo (id serial PRIMARY KEY, val text);
INSERT INTO vm_demo SELECT i, md5(i::text) FROM generate_series(1, 10000) i;

-- 2. Before vacuum: all-visible bits are NOT set
SELECT count(*) FILTER (WHERE all_visible) AS visible_before
FROM pg_visibility_map('vm_demo');
-- Expected: 0 (freshly inserted rows, no vacuum yet)
```

```

-- 3. Run vacuum
VACUUM vm_demo;

-- 4. After vacuum: bits should now be set
SELECT
    count(*) AS total_pages,
    count(*) FILTER (WHERE all_visible) AS visible_after,
    count(*) FILTER (WHERE all_frozen) AS frozen_after
FROM pg_visibility_map('vm_demo');
-- visible_after should equal total_pages (all clean)

-- 5. Trigger an UPDATE (clears VM bit for affected page)
UPDATE vm_demo SET val = 'modified' WHERE id = 1;

SELECT count(*) FILTER (WHERE NOT all_visible) AS non_visible_pages
FROM pg_visibility_map('vm_demo');
-- Should be 1 (the page containing id=1 was dirtied)

-- 6. Index-only scan before and after vacuum
CREATE INDEX ON vm_demo (id) INCLUDE (val);

EXPLAIN (ANALYZE, BUFFERS)
SELECT val FROM vm_demo WHERE id = 500;
-- Note Heap Fetches

VACUUM vm_demo;

EXPLAIN (ANALYZE, BUFFERS)
SELECT val FROM vm_demo WHERE id = 500;
-- Heap Fetches should be 0 now

-- 7. Check freeze status
VACUUM FREEZE vm_demo;
SELECT
    count(*) FILTER (WHERE all_frozen) AS frozen_pages,
    count(*) AS total_pages
FROM pg_visibility_map('vm_demo');
-- frozen_pages should equal total_pages after VACUUM FREEZE

```

Common Mistakes

1. **Assuming index-only scans are always used when an index exists.** The planner uses index-only scans only when the VM says the relevant pages are all-visible. After bulk inserts before autovacuum runs, heap fetches can be 100%.
2. **Not running VACUUM after bulk inserts.** A freshly loaded table has no all-visible bits set. Run `VACUUM ANALYZE` after any bulk load to set VM bits and update statistics.
3. **Confusing all-visible with all-frozen.** All-visible means no snapshots need old versions from this page. All-frozen means all XIDs have been replaced with FrozenXID. A page can be all-visible but not all-frozen (e.g., after regular VACUUM but before VACUUM FREEZE).

4. **Ignoring VM in bloat analysis.** A large VM (many non-visible pages) means VACUUM has to scan most of the table every run. This is a sign the table needs more frequent vacuuming.
5. **Believing `pg_relation_size(t, 'vm')` captures VM importance.** The VM file is tiny but its effect on performance is enormous. Do not dismiss it just because it is small.

Best Practices

- Run `VACUUM ANALYZE` after bulk inserts to set all-visible bits before queries hit the table.
- Use covering indexes (`INCLUDE` clause) to enable index-only scans on frequently accessed columns.
- Monitor Heap Fetches in `EXPLAIN ANALYZE` — high numbers mean the VM is stale and autovacuum is not keeping up.
- Consider `VACUUM FREEZE` on archival tables that will never change — all-frozen bits prevent any future freeze overhead.
- Use `pg_visibility` to verify VM health before and after maintenance windows.

Performance Considerations

Scenario	VM Impact	Action
Freshly inserted table (no vacuum)	All-visible bits clear → index-only scans degrade to heap fetches	<code>VACUUM ANALYZE</code> immediately after load
High-update table	VM bits constantly cleared → <code>VACUUM</code> must scan full table every time	Increase autovacuum frequency; consider partitioning
Read-heavy, rarely updated table	VM stays all-visible → <code>VACUUM</code> is extremely fast and index-only scans always work	Verify with <code>pg_visibility_map</code>
Archival/cold partition	<code>VACUUM FREEZE</code> → all-frozen → no future vacuum work	<code>VACUUM FREEZE</code> once after loading
Index-only scan showing Heap Fetches: N	VM bits not set	Run <code>VACUUM</code> and re-check

Interview Questions

Q1. What is the Visibility Map and where is it stored?

The VM is a per-relation data structure that stores 2 bits per heap page: the all-visible bit and the all-frozen bit. It lives in the `_vm` fork of the relation file (e.g., `16385_vm` alongside `16385`).

Q2. What does the all-visible bit mean, and what sets/clears it?

All-visible means all tuples on the page are visible to every active transaction — no running query needs to see an older version of any tuple on that page. `VACUUM` sets it after confirming this condition. Any modification (`INSERT`, `UPDATE`, `DELETE`, `HOT_UPDATE`) to the page clears it immediately.

Q3. How does the VM enable index-only scans?

An index-only scan reads indexed values from the index without fetching the heap. However, the index does not contain visibility information. The VM provides the visibility guarantee: if the heap page is all-visible, the executor can return the value from the index without checking the heap. If the page is not all-visible, the heap must be fetched to verify visibility.

Q4. What is the all-frozen bit and why is it separate from all-visible?

All-frozen means all tuples have `t_xmin = FrozenTransactionId`, so no future VACUUM freeze scan needs to visit the page. A page can be all-visible (no snapshots need old versions) without being all-frozen (some tuples still have real XIDs that will eventually need freezing). All-frozen is a stronger condition.

Q5. How large is the VM file for a 10 GB table?

A 10 GB table has approximately $10 \times 1024^3 / 8192 \approx 1,310,720$ heap pages. Each VM page covers 32,768 heap pages. So $1,310,720 / 32,768 \approx 40$ VM pages $\times 8 \text{ KB} = 320 \text{ KB}$. The VM is always tiny relative to the heap.

Q6. Why does `Heap Fetches` appear in `EXPLAIN ANALYZE` for an index-only scan?

`Heap Fetches > 0` means some index entries pointed to heap pages where the all-visible bit was not set, forcing the executor to read those pages to verify visibility. Typically caused by stale VM (table needs VACUUM).

Q7. What extension allows direct inspection of the VM?

`pg_visibility` (bundled with PostgreSQL). Key functions: `pg_visibility_map('table')` returns all-visible/all-frozen bits per page; `pg_check_visible('table')` verifies consistency between VM and heap page headers.

Q8. What effect does `VACUUM FREEZE` have on the VM?

It freezes all eligible tuples (replaces `t_xmin` with `FrozenTransactionId`) and sets the all-frozen bit in the VM for each page where all tuples are now frozen. Future VACUUM and freeze scans can skip these pages entirely.

Exercises with Solutions

Exercise 1 — Confirm VM bits enable index-only scans

```
-- Solution
CREATE TABLE iotest (id int PRIMARY KEY, email text);
INSERT INTO iotest SELECT i, 'user' || i || '@example.com' FROM generate_series(1, 100000) i;
CREATE INDEX iotest_email_idx ON iotest (id) INCLUDE (email);

-- Check VM before vacuum (bits not set)
SELECT count(*) FILTER (WHERE all_visible) AS visible FROM
pg_visibility_map('iotest');

-- Run explain - expect Heap Fetches > 0
EXPLAIN (ANALYZE, BUFFERS) SELECT email FROM iotest WHERE id BETWEEN 1 AND 1000;

-- Vacuum
VACUUM iotest;
```



```
-- Check VM after vacuum
SELECT count(*) FILTER (WHERE all_visible) AS visible FROM
pg_visibility_map('iotest');

-- Re-run explain - expect Heap Fetches = 0
EXPLAIN (ANALYZE, BUFFERS) SELECT email FROM iotest WHERE id BETWEEN 1 AND 1000;
```

Exercise 2 — Observe VM bit clearing on UPDATE

```
-- Solution
-- Continue from Exercise 1
SELECT blkno, all_visible FROM pg_visibility_map('iotest') LIMIT 5;
-- All should be true

UPDATE iotest SET email = 'changed@example.com' WHERE id = 1;

-- Page 0 (containing id=1) should now be false
SELECT blkno, all_visible FROM pg_visibility_map('iotest')
WHERE blkno = 0;

VACUUM iotest;

-- After vacuum, page 0 should be all-visible again
SELECT blkno, all_visible FROM pg_visibility_map('iotest')
WHERE blkno = 0;
```

Cross-References

- **06_vacuum.md** — VACUUM sets all-visible and all-frozen bits
- **07_autovacuum.md** — Autovacuum keeps VM bits current
- **09_free_space_map.md** — FSM tracks free space; VM tracks visibility
- **02_heap_storage.md** — PD_ALL_VISIBLE flag in page header mirrors VM
- **07_Indexes** — Index-only scans depend on VM